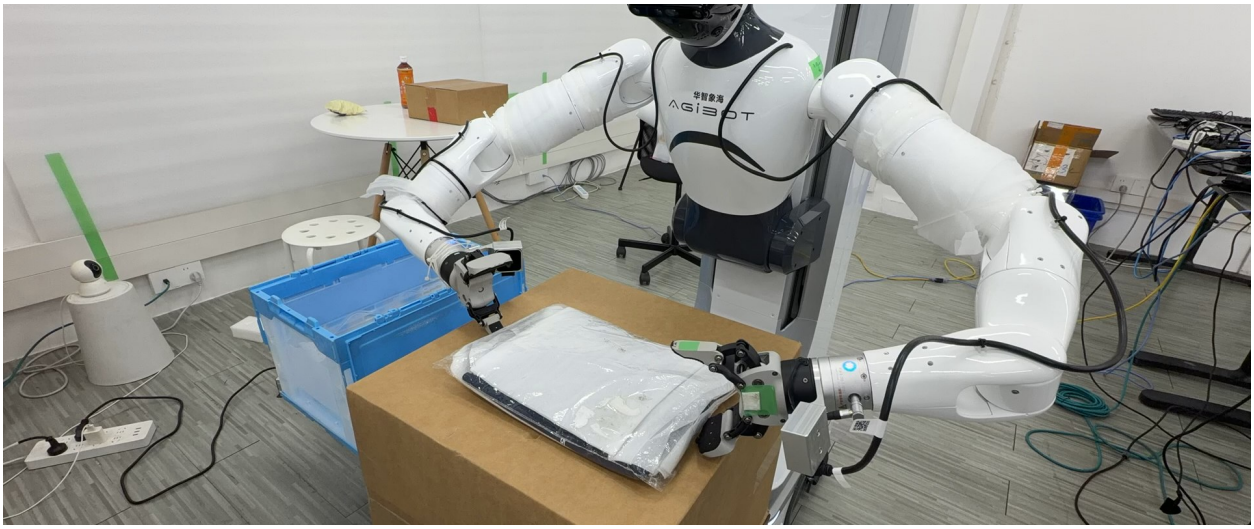
MUJIN · INTERNSHIP PROOF OF CONCEPT

Humanoid Robotics for Warehouse Parcel Handling

Imitation-learned dual-arm manipulation on the AgiBot Genie G1



The robot orienting a parcel so its shipping label faces upward.

Prepared by	Chen Yanyu (陈彦宇), Intern
Period	25 May – 31 July 2026 (10 weeks)
Date	31 July 2026

Contents

1. Executive summary

This project built a proof of concept to test whether a humanoid robot is a viable path for warehouse work, and whether it could integrate into MUJIN's systems in the near term.

The demonstration task is parcel label-orientation: the robot picks a parcel from a table, lifts it, determines whether the shipping label is visible, flips the parcel if it is not, and places it label-up for downstream scanning.

The system is built on the diffusion policy framework of Chi et al. [1], trained on 200 demonstrations collected through VR teleoperation on an AgiBot Genie G1. The manipulation behaviour is not programmed; it is learned by imitation from those demonstrations.

The system runs autonomously end to end, detecting and recovering from its own failed grasps and pausing safely when there is no work in front of it.

Summary

Question	Answer
Does it work?	Yes, for the demonstrated task under conditions resembling the training data. The full cycle runs autonomously with failure recovery.
Is it warehouse-ready?	No. Cycle time is slower than manual handling, though no baseline was measured, generalisation is untested, and the system works only from a fixed viewpoint (section 4.1).
What is new here?	The capability to teach a manipulation behaviour by demonstration rather than by programming, with no explicit modelling of the object or the motion.
What is the main risk?	Data cost, data quality, consistency and safety. Each new behaviour needs roughly 100 to 200 human demonstrations; demonstrations that suit a person do not necessarily suit the model; and a learned policy is inherently less repeatable than conventional motion planning.

Numbers

Measure	Result
Task success rate	~90%
First-grasp success rate	~ 53%
Median cycle time	~12 seconds

*(collected over 38 trials) rounded to the nearest whole number. Uncontrolled experiment, for reference only!

2. Demand analysis and goals

The brief was to demonstrate what a humanoid robot can do. Before designing that demonstration I had to decide what a humanoid would be for in a warehouse, since that determines which capabilities are worth showing.

2.1 Analysis of roles humanoid would serve in a warehouse scenario

Take over work still done by hand. Some warehouse labour remains manual because bespoke automation was never justified for it. A humanoid works in space and with tools built for people, so adopting one would not require re-engineering the site.

Handle what conventional automation cannot. High SKU variety, deformable and irregular items, and tasks needing two hands and a judgement call. Conventional systems need a visual model per object class and a motion plan per task, and that cost recurs with every new scenario.

Reach where fixed equipment cannot. AGVs are large, and a fixed arm has a fixed workspace. A robot of human size and reach could work in narrow aisles, at shelving, and in the corners of a facility laid out around people.

2.2 What that implies for the demonstration

Each role rests on a capability rather than a task, so the demonstration had to show the capability: perceiving from the robot's own cameras rather than fixture-mounted sensors, using two arms on objects with no fixed geometry, and acquiring the behaviour without per-task engineering, since otherwise setup cost moves rather than falls.

The third role could not be demonstrated. Moving the wheelbase would have meant solving mobility on top of perception, manipulation and control, which ten weeks did not allow. Mobility is argued here, not shown.

2.3 Goals

A. Perform a complete warehouse task autonomously, using only the robot's own cameras and both arms. Establishes that a humanoid can do useful work without the fixtures and sensors a conventional cell requires.

B. Acquire the behaviour by demonstration rather than by programming. No object model, no grasp planner, no hand-written trajectory. This determines whether setup cost falls for each new scenario.

C. Handle a task that is difficult to script, including its failures. A deformable object, a decision that changes what the robot does next, and autonomous recovery from a failed grasp: coping with a scenario rather than replaying a motion.

3. What was built

The system consists of three layers: demonstration, policy execution, and control and safety. An operator first teaches the robot through demonstrations. A learned policy then reproduces the task behaviour, while the control and safety stack converts policy outputs into executable robot motion. Six components implement these layers.

3.1 Teaching by demonstration

An operator wearing a VR headset controls both robot arms while performing the task. Three cameras (one mounted on each hand and one on the head), together with robot joint states, are recorded and synchronised with timestamps.

This allows a warehouse engineer to teach new behaviours without modifying code. However, each new behaviour requires approximately 100–200 successful demonstrations, and the final policy performance is limited by demonstration quality. Section 5.3 discusses this trade-off.

3.2 Policy server

A diffusion policy receives three camera views, end-effector poses, and gripper states, and predicts a short horizon of future end-effector targets. The policy runs at 10 Hz on a separate inference server.

The policy controls the grasp, lift, and flip stages, which involve complex contact interactions that are difficult to model with scripted trajectories. The training dataset contains approximately 200 demonstrations.

Placement, release, recovery, and start/stop conditions are handled by scripted routines triggered by perception outputs. This hybrid design is intentional: placement is geometrically constrained and repeatable, making scripted motion more reliable and requiring significantly less data than learning-based approaches. The limitation is that new placement locations require additional scripting.

3.3 Perception

The perception system provides three outputs: package presence, label visibility, and grasp success. A single locally deployed detector is used for all three tasks.

Before adopting the learned detector, three rule-based approaches were evaluated: barcode reading, fiducial markers, and printed text-block detection. None achieved sufficient reliability for triggering robot actions. The learned detector provided higher reliability and reached deployment readiness faster. This observation is documented in the project knowhow document.

3.4 From policy output to arm commands

The policy produces end-effector targets at 10 Hz, while the robot controller operates at 120 Hz. This layer bridges the frequency gap and was the largest engineering effort in the project.

Each predicted waypoint is divided into smaller motion increments, which can be executed individually, in batches, or continuously. Consecutive predictions are combined through temporal ensembling and

stored in a buffer. The entire buffer is then smoothed, and the controller executes the smoothed trajectory rather than individual predictions.

Prediction and execution alignment is maintained using an identifier based on the robot control step count instead of wall-clock time. This prevents timing errors caused by variable inference latency or execution speed changes. Commanded velocities and joint angles are also ramped across trajectory segments to reduce discontinuities. Section 5.2 discusses the challenges encountered in this component.

3.5 Safety guards

The policy does not directly command the robot. Policy outputs are passed through inverse kinematics, collision validation in simulation, and motion limit checks before being sent to the robot. Commands are streamed as bounded increments with velocity and step-size limits. A watchdog immediately stops motion when abnormal commands are detected.

Additional safety checks monitor sudden joint changes and workspace violations. A retreat primitive is available to return the robot arms to a predefined safe pose.

Replacing the vendor inverse kinematics solver. The vendor SDK provides a closed-source inverse kinematics solver. It was replaced with a transparent solver based on the published robot model. Validation against recorded robot motion across more than 400 target points showed a maximum deviation below 8 mm and 0.03 rad. The same solver is used in both simulation and hardware, enabling pre-execution validation.

3.6 Development framework and interface

The vendor SDK is isolated behind a single interface boundary. The kinematics, simulation, and timing layers are implemented independently. This design allows future replacement with another robot platform supporting joint-angle control without major architectural changes, although this has not been tested.

The same separation enables simulation, evaluation, and interface development on a standard laptop without requiring the physical robot, SDK, or ROS. As a result, most debugging and development could be performed without robot access.

The interface also acts as the system launcher: the complete stack starts from a single command. Camera streams can be enabled independently, and all operating parameters (including teleoperation sensitivity, detector thresholds, and motion limits) can be adjusted during runtime and saved without restarting. This is important because system tuning requires many iterative parameter adjustments, and restarting after each change would significantly slow development.

3.7 System capabilities

Capability	At handoff
Autonomous full cycle: grasp, lift, flip decision, place	Runs repeatedly without operator input under conditions resembling the training set.
Label-based flip decision	The robot detects whether the waybill is visible and selects the flip or no-flip placement accordingly.
Grasp failure detection and recovery	A three-class gripper-state detector distinguishes an empty closed gripper from a successful grasp; the robot clears its queue, opens, homes, and re-attempts.
Idle gating	Inference pauses and the arms park when no parcel is present. Deliberately fails open: it will not stop the robot if detection breaks, which is correct for a demo and wrong for production.
Safety envelope	Velocity and step bounds, watchdog stop, simulation collision validation, joint-jump and workspace monitors, data-driven retreat.

3.8 Goals

Against the goals set in section 2.3:

Goal A was met, in that the full cycle runs autonomously from the robot's own cameras using both arms.

Goal B was met in part; the contact-rich portion of the task was acquired by demonstration, but placement, release and recovery remain scripted.

Goal C was met, in that the task involves a deformable object, a decision that changes what the robot does next, and autonomous recovery from a failed grasp.

4. Limitations

This section covers what the system does not do, and what was not tested. Section 5 provides an assessment of what these limitations mean.

4.1 The system works only from a fixed viewpoint

Diffusion policy in its current form requires the head camera to hold a fixed pose, because the learned visual mapping is invalidated if the viewpoint moves. In practice this makes the humanoid a fixed-base system that can work only while standing still. Moving the wheelbase would invalidate that mapping, and would additionally introduce localisation, navigation and stability as new problems.

4.2 Generalisation is untested

Every result used parcels and starting positions resembling the training set, under consistent lighting. Behaviour on a novel package, an off-distribution starting pose or changed lighting was not measured.

4.3 Data cost

Reaching reliable behaviour took roughly 100 to 200 demonstrations. Robustness improved with volume, and returns diminished above roughly 100 training epochs. Whether that cost is repeated in full for every new behaviour, or amortises across behaviours, was not established.

4.4 Narrow recovery

Recovery handles a failed grasp. It does not handle a parcel dropped mid-flip, a misplaced parcel, or a mechanical jam. No broader failure taxonomy was implemented.

5. Discussion

This chapter summarizes the key insights gained from designing, implementing, and operating the system over the ten-week project. These observations are drawn from engineering practice rather than controlled experiments. Sections 5.1–5.3 discuss the strengths, challenges, and limitations of the proposed approach, while Sections 5.4 and 5.5 provide an overall assessment of humanoid manipulation and highlight several unresolved questions.

The main conclusions are as follows:

- **Can a learned policy acquire implicit behaviors that would otherwise require manual programming in conventional control systems?** — **Yes.** The robot consistently reorients the package before releasing it and executes a carefully coordinated retreat motion, both of which emerged from learning rather than explicit programming. These behaviors are discussed in Section 5.1.

- **Can demonstrations from skilled operators reduce deployment costs in new scenarios?** — **Yes,** but with important caveats. Skilled task execution does not necessarily produce training data suitable for imitation learning. The quality and consistency of demonstrations remain critical, as discussed in Section 5.3.

- **Can humanoid robots be effectively controlled using traditional control methods alone?** — **No.** Humanoid robots are substantially more difficult to control than conventional industrial manipulators. Learning-based control is therefore not merely an enhancement but a prerequisite for making humanoid robots practical, as argued in Section 5.4.

- **Are current humanoid hardware and neural network control technologies mature enough for warehouse deployment?** — **No.** Present-day humanoid robots remain expensive, slower than conventional automation, and achieve task success rates of approximately 90%, well below the reliability required for industrial deployment. Further discussion is provided in Section 5.4.

- **Are industrial warehouses a natural first application for humanoid robots?** — **Uncertain.** Most warehouse and factory operations are already served by highly effective conventional automation. Tasks that remain manual are often retained for economic rather than technical reasons, raising questions about the near-term value proposition of humanoid robots in such environments. This issue is explored further in Section 5.5.

5.1 Learning tacit manipulation details

The strongest result of the project was not task completion itself, but the ability of the policy to reproduce manipulation details that were never explicitly specified. For example, after flipping a parcel, the policy learned to orient the parcel before releasing it. If the orientation could not be completed, the robot delayed withdrawal rather than immediately releasing. During withdrawal, it also reproduced a small acceleration used by human operators to overcome surface friction.

Neither behaviour was programmed. Conventional control can reproduce these actions, but only after an engineer identifies the behaviour and manually encodes it. The diffusion policy instead extracted these patterns from repeated demonstrations.

This suggests that the value of imitation learning may lie less in replacing individual motion planning problems, and more in reducing the engineering effort required to transfer skilled manipulation behaviours to new tasks.

5.2 The main challenge: converting inference into reliable motion

The hardest problem faced during the project was converting inference into motion that is smooth and consistent. Several sources of timing error compound here. Inference latency varies from one prediction to the next, which introduces lag that cannot be anticipated. Camera frames reach the policy through a cache layer and therefore lag the arm state they are meant to describe. Synchronising against wall-clock time makes both problems worse, because it assumes a fixed relationship between elapsed time and robot state that does not hold.

The solution was to stop synchronising in real time altogether. Each trajectory now carries a master identifier tied to the robot's own step count, and every inference result is stamped with the step it was computed from. Alignment is then true to the state of the robot rather than to the clock, and it survives variation in both execution speed and inference latency.

5.3 Data quality is not equivalent to operator experience

A key limitation discovered during data collection was that skilled human operators do not automatically produce optimal training data.

The reason is that people optimise for effort. With enough practice a person performs a task using the smallest movements that still succeed, and for the person that efficiency costs nothing in reliability. A neural network learning the same action has no such experience to draw on. It behaves like a beginner, and it needs clearly identifiable boundaries between the phases of an action. Where an expert's movements blur those boundaries, learning becomes markedly less effective.

In practice this meant deciding in advance which characteristics of the action I wanted the model to acquire, and then deliberately exaggerating them while demonstrating. Skill at the task and skill at teaching the model are therefore not the same skill. This qualifies the deployment argument in 5.1: an experienced worker shortens the path to a competent model, but only if the demonstrations are shaped for the model rather than performed as the worker would naturally perform them.

5.4 Assessment

Humanoid hardware and learned control should be considered together. Humanoids provide a human-compatible form factor, but their increased degrees of freedom and dynamic complexity make conventional control approaches significantly more difficult than those used for industrial arms.

Neural-network control is therefore likely to be a requirement for practical humanoid operation rather than merely an optimisation.

However, neither technology is currently ready for general warehouse deployment. The demonstrated system achieved approximately 90% task success under conditions similar to the training data, which remains below the reliability expected for autonomous industrial operation.

5.5 Concluding Remarks: Open Questions

Two unresolved questions remain, each of a very different nature.The first is a technical issue arising from insufficient testing rather than any fundamental limitation. As this technology is still in its early stages, experimental validation has been limited, leaving many aspects of the system's behavior unexplored. This is characteristic of emerging technologies, and these unknowns are likely to diminish as the field continues to mature.

The second issue was far less expected. As the project progressed, I became increasingly uncertain about the practicality of humanoid robots in industrial environments. Over the past two to three decades, automation has advanced to the point where many factories require little or no human intervention. It is difficult to identify industrial tasks for which no automation solution already exists. In most cases, human labor remains only because it is currently more cost-effective, not because automation is technically infeasible. Consequently, humanoid robots must compete against established systems that are already deployed, operate faster, and have been proven reliable over many years, making their advantages less compelling.

This raises a broader question: rather than industrial applications, might humanoid robots first achieve widespread adoption in domains that demand robust operation in highly unstructured environments but currently have relatively low levels of automation—such as household settings? This is a question that deserves careful consideration.

6. Development trajectory

The ten weeks divide into four phases. The shape of the progression is itself informative: roughly half the elapsed time went into motion quality and timing, comparatively only small efforts were needed to train the model.

Weeks	Phase	Outcome
1–2	Foundation	Reproduced the reference diffusion policy, built a custom data collection system, chose the state representation, established the end-to-end pipeline in simulation.
3–5	Single-arm autonomy	First autonomous execution on hardware. Most of this period was spent diagnosing oscillation (the arm stalling and rocking in place), which was ultimately a timing and trajectory-merging problem, not a model problem.
6–7	Dual-arm and recovery	Dual-arm data collection and control, the flip task, scripted placement, automatic homing, and grasp failure recovery.
8–10	Robustness and vision	Safety monitors, demo packaging, evaluation logging, and the migration of all perception onto a single learned detector.

References

[1] C. Chi et al., "Diffusion policy: Visuomotor policy learning via action diffusion," Int. J. Robot. Res., vol. 44, no. 10–11, pp. 1684–1704, 2025.

[2] J. Varley et al., "Embodied AI with two arms: Zero-shot learning, safety and modularity," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), Abu Dhabi, UAE, Oct. 2024, doi: 10.1109/IROS58592.2024.10802181.

[3] Y. Jin and B. Haworth, "Back to basics: Motion representation matters for human motion generation using diffusion model," arXiv:2512.04499, 2025.

Appendix: Weekly progress summary

Week	Dates	Work completed
1	25–29 May	Reproduced the reference diffusion policy end to end on a benchmark task to validate the training and inference workflow. Established from the literature that end-effector pose outperforms joint angles as a training representation, and that a 6D rotation representation is preferable to quaternions because it is continuous. Began building a data collection system capturing camera, end-effector pose and gripper state together. Traced a fault where recorded end-effector position never updated to the depth camera stream consuming roughly 600 MB of bandwidth and starving the rest of the system.
2	1–5 June	Settled the preprocessing strategy: leave observations exactly as the cameras recorded them and smooth the action stream instead, so camera frames and pose data stay aligned. Corrected quaternion sign inversions introduced at recording time. Collected the first 20 single-arm demonstrations and trained the first working model. Restructured the codebase around a central environment layer to eliminate thread contention. Validated a Pinocchio-based inverse kinematics solver against recorded motion, with maximum deviation under 8 mm and 0.03 rad across more than 400 target points.
3	8–12 June	Added the safety layer: a per-command joint change limit of 0.02 rad and an emergency stop acting within 30 ms. Ran the first complete pipeline test on hardware and achieved first fully autonomous execution. Introduced substep control, subdividing each predicted waypoint into increments that can be released individually, in batches, or continuously. Normalised and binarised the gripper command. Both trajectory-merging strategies implemented produced oscillation, in which the arm stalls and rocks in place rather than progressing.
4	15–18 June	Collected 50 demonstrations and added joint angles alongside end-effector pose as model input, compensating for the model's limited awareness of arm configuration. Implemented temporal ensembling across successive predictions. Identified that cropping camera input to 84×84 was both too small and distorting the head camera's aspect ratio; retrained at higher resolution with a separate visual encoder per camera, so the head camera handles coarse positioning and the wrist cameras handle alignment. Logging every prediction revealed that entire predicted trajectories sometimes fell below the preceding one, the direct cause of the oscillation.
5	22–26 June	Found the end-effector pose reported by the SDK was frozen while joint angles continued updating, and switched to deriving pose through forward kinematics instead, which produced a marked improvement. Replaced explicit trajectory merging with a buffer that accepts each new prediction on arrival and is smoothed as a whole. Rebuilt the timing system around absolute trajectory identifiers rather than wall-clock synchronisation, so

Week	Dates	Work completed
		alignment survives variable inference latency. Corrected a velocity scaling fault that left acceleration segments running faster than intended. Extended the prediction horizon from 8 to 14 steps.
6	29 June – 3 July	Built the dual-arm control and data collection infrastructure, including fine end-effector adjustment and wrist rotation for the flip motion. Collected 100 dual-arm flip demonstrations and trained the dual-arm model. Retrained at 10 Hz after a 30 Hz model proved too slow to infer. Identified a mismatch between two trajectory generators, one subdividing by joint angle change and the other by fixed step count, as the remaining source of jitter.
7	6–10 July	Corrected the trajectory generation mismatch, which removed the abrupt acceleration between prediction segments. Integrated grasp failure detection and recovery into the autonomous loop, with automatic homing before each run and a reset of the merge buffer afterwards. Improved grasp reliability through a reach-then-close strategy, a forward adjustment before closing, and a convergence step aligning the two arms. Changed the placement trigger from a fixed wrist angle threshold to a change in wrist angle after grasping, which is robust to variation in grasp pose.
8	13–19 July	Packaged the control interface for demonstration and added live experiment logging and an evaluation panel. Added recovery from inverse kinematics failure, which had previously left the arm stalled. Added joint discontinuity and end-effector workspace monitors, with retreat to the nearest of five safe positions rather than a single home pose. Extended the training set to 200 episodes, deliberately including harder grasp conditions.
9	20–24 July	Developed the no-flip placement path, so a parcel arriving label-up is placed without unnecessary reorientation. Label detection went through four approaches: barcode reading, too noisy to trigger an action; fiducial markers, used only to validate the pipeline; printed text-block detection, worse than barcodes despite substantial tuning; and finally a trained detector, which was markedly more stable. Consolidated parcel presence, label detection and three-class gripper state onto a single detection model, with a commit latch requiring sustained detection before any macro fires.
10	27–31 July	Structured evaluation, documentation and handover: this report, the knowledge-base entry, the evaluation protocol, and archiving of models, datasets and calibration.

Appendix: Policy-to-execution framework